

Distributed Symmetric Key Establishment with Forward Secrecy for Implantable Medical Devices

Roozbeh Sarenche , Sayon Duttagupta , Francesco Milizia, Kevin Bogner ,
and Varesh Mishra 

COSIC, KU Leuven, Leuven, Belgium
`firstname.lastname@esat.kuleuven.be`

Abstract. Implantable Medical Devices (IMDs) operate for many years in an ecosystem where device loss, backend compromise, and physical capture are realistic long-term threats. While prior work has extensively studied secure pairing and access control, existing IMD architectures typically rely on long-lived secrets. As a result, the compromise of a single credential can retroactively expose years of sensitive patient telemetry. Limiting such damage requires Perfect Forward Secrecy (PFS), yet achieving PFS in IMDs is challenging due to strict energy constraints, intermittent connectivity, and safety requirements. Public-key cryptography is often too costly for frequent use on implants and physical-layer defences rely on fragile assumptions. In this work, we present a symmetric-only communication protocol that provides forward secrecy for IMDs while remaining robust to device loss and state desynchronisation. The protocol evolves cryptographic state across sessions using lightweight primitives on the implant, ensuring that past communications remain confidential even if the IMD or associated client devices are compromised later. To address the safety risks of desynchronisation, we introduce a threshold-assisted recovery mechanism that enables secure resynchronisation via a distributed backend without trusting any single client or server. The design preserves patient-centric control and supports controlled emergency access without undermining long-term confidentiality. We formally analyse the protocol using ProVerif and demonstrate feasibility through implementations on a 16-bit MSP430 and a 32-bit ARM Cortex-M33 microcontroller.

1 Introduction

Implantable Medical Devices (IMDs), such as pacemakers, cardiac defibrillators and neurostimulators, enable continuous sensing and therapy over many years, creating a uniquely demanding security setting. The device is physically embedded and power-constrained, yet wireless access is essential for clinical follow-up, configuration, and emergency care. The surrounding ecosystem also evolves repeatedly over the device lifetime: patients lose or replace phones, clinician workstations are compromised, hospital infrastructure faces ransomware, and devices

may ultimately be explanted and analysed. Prior work has demonstrated both the reality of wireless attacks and the fragility of deployed protections, from software radio attacks and low power defences [6] to repeated demonstrations of insecurity in modern cardiac and neurostimulation systems [10,12] and practical attacks on insulin therapy systems [8].

This long-horizon view exposes a gap that is easy to miss if security is framed only as a “moment of access.” Much of the literature focuses on bootstrapping access or proximity-based protection. Shielding approaches aim to prevent eavesdropping by controlling the physical channel [5,19]. Physiological keying aims to extract keys from the body itself [7,15]. Delegation-based systems place access control and key management in a companion device such as a phone [4]. These directions address important clinical constraints, especially emergency access and usability [2,3]. However, they do not directly solve a different and increasingly dominant risk: *retrospective compromise*. If long-lived secrets are reused, a compromise years after deployment can retroactively expose previously recorded telemetry and commands. If a protection relies on physical or biological assumptions, later adversaries with stronger sensing capabilities may violate those assumptions, as shown for friendly jamming under stronger receiver models (MIMO) [18] and for remote recovery of heartbeat signals using cameras [17]. Even when physiological features are used, their instability and limited entropy complicate robust key establishment under realistic conditions [9]. The result is a tension between what is clinically necessary and what is cryptographically desirable: we require the damage containment properties of Perfect Forward Secrecy (PFS), but we cannot assume that the implant can afford frequent public key operations, nor can we assume that patients will consistently carry extra hardware.

This paper reframes IMD security as a *session lifecycle* problem rather than a *pairing* problem. Our goal is to provide strong long term confidentiality for IMD communications even when compromises occur late, while preserving the constraints that make IMDs hard: low energy budgets, intermittent connectivity, message loss, emergency workflows, and changing stakeholders over time. Crucially, our setting also demands patient agency: outside true emergencies, patients should retain meaningful control over who can access their health data, rather than delegating permanent authority to a backend.

To address these requirements, we design *blablablablablablablab*, a symmetric-only protocol that evolves device state across sessions and uses a threshold set of backend servers to assist resynchronisation and recovery. Building upon the symmetric state-evolved PFS introduced in the SAKE protocol [1], we achieve the properties without the heavy computational cost of Diffie-Hellman. The design distributes trust so that no single compromised phone, clinician device, or backend component becomes a perpetual point of failure. At the same time, it supports practical clinical workflows, including controlled and auditable break-glass access, consistent with prior observations about safety and human values in IMD ecosystems [2,3,14].

Contributions: To summarise, this paper makes the following contributions.

- **A symmetric-only forward-secure protocol for IMDs.** We design a lightweight, state-evolving protocol that provides forward secrecy for implantable medical devices using only symmetric primitives on the implant. Past sessions remain confidential even if the IMD or associated client devices are fully compromised at a later time.
- **Threshold-assisted resynchronisation.** We introduce a recovery mechanism that enables secure resynchronisation after message loss, desynchronisation, emergency, or client device replacement by leveraging a threshold of backend servers. This avoids permanent lockout and removes any single point of failure while preserving the implant’s symmetric-only execution model.
- **Full-stack implementation on constrained and modern platforms.** We implement the protocol on a 16-bit MSP430 microcontroller and a 32-bit ARM Cortex-M33 microcontroller, representing both legacy academic platforms and contemporary industry-grade devices. We further deploy the backend threshold logic across multiple servers to demonstrate end-to-end feasibility in a realistic distributed setting.
- **Formal verification under state compromise.** We formally analyse the protocol using ProVerif, modelling an adversary with full network control and long-term state compromise capabilities. We establish secrecy and authentication guarantees in the presence of state evolution and recovery.

2 Background and Clinical Safety Model

This section provides the clinical and system context used throughout the paper. Our goal is not to survey IMDs in full detail, but to make explicit the operational realities that drive the security requirements, especially the need to avoid lockout in emergencies while still protecting long-term confidentiality.

2.1 IMD ecosystem and communication model

We consider an IMD ecosystem consisting of an implant, one or more external client devices, and a clinical backend. The implant communicates wirelessly for telemetry retrieval, configuration, and therapy-related operations. In practice, IMDs use short-range radio technologies, including proprietary links and standards such as BLE and NFC. In our design and prototype, we focus on BLE as a representative low-power radio interface. The external client device can be a patient smartphone used for day-to-day interactions and consent, or a clinician programmer used during clinical follow-up. The backend represents a hospital or a federation of clinical operators that provides authorisation, auditing, and recovery support. This ecosystem is long-lived and dynamic: devices are replaced, clinical staff changes, and backend services may be temporarily unavailable. Any realistic security mechanism must remain safe and usable under these conditions [16].

2.2 IMD operation taxonomy

We group IMD interactions into three broad classes based on clinical risk and required timeliness.

Routine telemetry. These operations collect device status, physiological measurements, and diagnostic logs for monitoring and follow-up. They are frequent, delay-tolerant, and primarily confidentiality-sensitive.

Non-urgent configuration. These operations change parameters such as thresholds, sensing windows, or reporting rates during planned clinical sessions. They may be integrity-critical and require strong authentication, but they are typically not time-critical to the second.

Urgent or emergency operations. These operations must be executable during emergencies or safety-critical episodes. Examples include actions required to restore safe device behaviour or disable harmful stimulation. For these operations, the dominant requirement is patient safety. The system must avoid permanent lockout even under desynchronisation, loss of the patient phone, or partial backend failures.

This taxonomy motivates a security model in which not all operations are treated equally. Strong confidentiality and integrity are required throughout, but the system must also provide a clinically acceptable fallback path for urgent operations [2].

2.3 Safety model and clinical fallback

Security for IMDs must coexist with safety requirements. In particular, emergency access is a first-class requirement, and it cannot depend on a single external artefact that may be missing when needed [3]. We therefore explicitly model clinical fallback as part of the system design. At a high level, the safety model requires that: (i) the implant remains operable under loss or replacement of external client devices; (ii) recovery from desynchronisation is possible without invasive procedures; and (iii) emergency access can be performed in a controlled and auditable manner.

We operationalise these requirements by associating each operation class with a security requirement and an acceptable clinical fallback, as summarised in Table 1.

2.4 System constraints

We highlight the constraints that shape the protocol design.

Energy and computation. IMDs operate within strict power budgets and are expected to function over many years. This limits the feasible frequency of expensive cryptographic operations on the implant and motivates a symmetric-only critical path, as asymmetric operations on standard IMD microcontrollers (e.g., MSP430) are prohibitively expensive [11, 14].

Wireless link characteristics. Short-range radio links are intermittent and lossy due to body effects, posture, interference, and mobility. Message loss and

Table 1: Operation taxonomy, security requirements, and clinical fallback expectations.

Operation class	Primary security requirement	Clinical fallback expectation
Routine telemetry	Confidentiality of patient data and session keys; integrity of reports	Delay is acceptable; recovery from transient failures should be automatic
Non-urgent configuration	Strong authentication and integrity; auditability of changes	Proceed only under explicit authorisation; defer if backend or policy checks fail
Urgent or emergency operations	Availability and safe execution; controlled access with accountability	No permanent lockout; allow a break-glass path with audit trails

reordering must be treated as normal rather than exceptional. Protocol state must therefore tolerate desynchronisation and support safe recovery.

Device churn and multi-client access. A patient may replace their phone, and multiple clinicians may need access over the device lifetime. The security model must support changes in external devices without forcing re-implantation, and without making any single client device a permanent trust anchor [4].

Auditability and accountability. Clinical settings require traceability of configuration changes and emergency interventions. The security architecture should support logging and auditing of sensitive operations, especially those performed under fallback conditions.

Patient agency. Outside true emergencies, the patient should retain meaningful control over who may access their IMD data. In our design, the smartphone acts as a consent and policy enforcement point, but the protocol must also remain safe when the phone is unavailable, which is unavoidable in practice [2].

2.5 Design implications

The above model leads to two design implications that guide the remainder of the paper. First, protecting long-term confidentiality requires forward secrecy under realistic compromise scenarios, but the implant must avoid frequent public key operations due to energy constraints. Second, any state-evolving design must include an explicit recovery mechanism, because desynchronisation is not merely a usability issue in this domain but a safety concern. Our protocol is designed with these requirements as first-class goals.

3 Threat Model

In this section, we first describe the system architecture and types of entities present in the system. Next, we discuss the capabilities of the assumed adversary and associated security goals.

3.1 System Model

We now discuss the roles of the entities within the system:

- **Trusted Authority ($\mathcal{TA}$)**: The Trusted Authority ($\mathcal{TA}$) is involved only during the system bootstrapping and key provisioning process. It generates system parameters, identity credentials and initial key material for sensors, distributes secret shares of initial key material and certifies the public keys of mobile devices. It is assumed that $\mathcal{TA}$ is fully trusted during its online phase and its operations are not compromised during initial provisioning of key materials. It can be assumed that after initial setup, ($\mathcal{TA}$) does not participate in the protocol related to key derivation further.
- **Implantable Medical Device (IMD) / Sensor ($\mathcal{S}_i$)**: The $\mathcal{S}_i$ stores its own key material and performs a lightweight symmetric protocol for key update and session key establishment. We assume that $\mathcal{S}_i$ executes the protocol honestly.
- **Key Servers ($\mathcal{KS}$)**: The Key Servers are the entities responsible for storing per sensor state and facilitating the derivation of the session key collaboratively. Let n be the number of key servers; $\mathcal{KS}_j$ denotes server j , and $\mathcal{KS}$ denotes the set of all n servers. We assume an honest majority amongst the set $\mathcal{KS}$ such that at most $(t - 1)$ of them can be corrupted and controlled by an adversary. Once a key server is corrupted, the adversary can read all sensor state information stored on that server.
- **User's Mobile Device (M^{user})**. The protocol assumes that each individual with an IMD possesses a personal mobile device. The user's mobile device enables the user to explicitly consent to or deny session-key establishment requests initiated by a doctor's device for the corresponding IMD. It stores the cryptographic material required for the user-side contribution to session-key derivation. The protocol further accounts for scenarios in which M^{user} is unavailable or unreachable, which are addressed through a dedicated emergency-access mechanism.
- **Doctor's Device (D^{Dr})**. The doctor's device D^{Dr} facilitates secure session establishment between a doctor and an IMD. It may initiate a session-key establishment request only if it possesses a valid certificate issued by the trusted authority $\mathcal{TA}$ and, in the non-emergency setting, obtains explicit consent from the user's mobile device M^{user} . The identity of D^{Dr} is authenticated through its certified public key.
- **Replicated Database/Ledger ($\mathcal{DB}_{\text{rep}}$)**: The Replicated Database $\mathcal{DB}_{\text{rep}}$ stores the identifier and update counters for each sensor $\mathcal{S}_i$. It also stores related certificates for public keys and associated metadata. We assume that

$\mathcal{DB}_{\text{rep}}$ provides integrity and append-only functionality. The adversary can read all the entries in the $\mathcal{DB}_{\text{rep}}$ but cannot delete or modify the committed entries and also cannot alter the state of the database to one of its previous state.

3.2 Adversary

The protocol is analyzed under a strong network adversary $\mathcal{A}$ operating in the Dolev–Yao model. The adversary has full control over the communication network and can eavesdrop on, block, delay, replay, reorder, and inject arbitrary messages exchanged between any system entities. Additionally, $\mathcal{A}$ can corrupt at-most $(t - 1)$ key servers and get access to the internal state of the corrupted key servers. The $\mathcal{A}$ cannot (i) break the assumed cryptographic primitives (ii) compromise and corrupt t or more key servers (iii) compromise $\mathcal{TA}$ operations during setup and key provisioning. The side channel attacks on IMDs are considered out of scope due to the nature of IMD deployment.

3.3 Security Goals

We define the security goals of our protocol as follows:

- **Authenticated Session-Key Establishment.** Upon successful completion of the protocol, the doctor’s device D^{Dr} and the sensor S_i derive a shared session key k_{session} , and each party is assured of the identity of its peer.
- **Session-Key Confidentiality.** The established session key k_{session} is known only to the intended endpoints, namely the sensor S_i and the doctor’s device D^{Dr} . The adversary, including a curious but honest minority of key servers, a curious user mobile device, other doctors, or external parties, learns no information about the session key.
- **Forward Secrecy.** As long as at least t key servers are online, the compromise of any protocol entities, including the sensor, mobile devices, and any subset of key servers, at time T does not enable the adversary to derive any session key established prior to time T .
- **User Consent and Emergency Access Control.** In non-emergency operation, session-key establishment requires explicit user consent via the mobile device M^{user} . When M^{user} is unavailable, emergency access is enabled through a threshold-based mechanism among the key servers, providing controlled access without user participation while preserving the protocol’s security guarantees.

4 Protocol Specification

In our protocol construction, we assume the existence of four categories of pre-shared keys: the synchronisation key, the user–sensor key, the server–sensor key,

and the emergency-access key. Except for the user–sensor key, which is permanent, all other keys are updatable and may be refreshed during the session-establishment process, depending on the session’s progression.

- **Synchronization key** k_i^{sync} : This key is shared for each sensor S_i between the sensor and the key servers $\mathcal{KS}$. It is used to maintain synchronisation of the key-update state across these entities.
- **User–sensor key** $k_i^{\text{U-S}}$: This key is shared between sensor S_i and the user’s mobile device M_i^{user} . It provides the user-side contribution to the session-key generation process.
- **Server–sensor key** $k_i^{\text{KS-S}}$: This key is shared between sensor S_i and the key servers $\mathcal{KS}$. It provides the servers’ contribution to the session-key generation process and is frequently updated to ensure forward secrecy.
- **Emergency-access key** $k_i^{\text{emergency}}$: This key is shared between the key servers $\mathcal{KS}$. It is used exclusively when the user’s mobile device is unavailable and clinical emergency access is required to reconstruct the user’s contribution to the session key without the participation of the user’s mobile device.

4.1 Protocol Preliminaries

Secret-Sharing Scheme $\mathcal{SC}$ The scheme $\mathcal{SC}$ is instantiated as a (t, n) threshold secret-sharing mechanism. It takes as input a secret key $k \in \{0, 1\}^\lambda$ and outputs n shares such that any subset of at least t key servers can reconstruct k , whereas any set of fewer than t shares reveals no information about the secret.

Let P be a public prime modulus satisfying $P > 2^\lambda$. The key k is mapped to an element $s \in \text{GF}(P)$ by interpreting k as an integer in the range $[0, 2^\lambda - 1]$. For each input k , $\mathcal{SC}$ samples coefficients $r_1, \dots, r_{t-1} \xleftarrow{\$} \text{GF}(P)$ and defines the polynomial

$$f(x) = r_{t-1}x^{t-1} + r_{t-2}x^{t-2} + \dots + r_1x + s \bmod P . \quad (1)$$

Each share is computed as $\mathcal{SC}(k, j) = f(j)$. Reconstruction is achieved via Lagrange interpolation over $\text{GF}(P)$.

Key Update To provide forward secrecy, the protocol employs a cryptographically secure one-way key-update function $\text{Upd}(\cdot)$, for which inverting $\text{Upd}(x)$ is computationally infeasible.

Sensor Key Update. For sensor S_i with key set

$$\mathcal{K}_i^{\text{sensor}} = \{k_i^{\text{sync}}, k_i^{\text{U-S}}, k_i^{\text{KS-S}}, r_i^{\text{S,emg}}\},$$

the key update is performed as follows:

- $k_i^{\text{sync}} \leftarrow \text{Upd}(k_i^{\text{sync}})$
- $k_i^{\text{KS-S}} \leftarrow \text{Upd}(k_i^{\text{KS-S}})$

Key-Server Update. For key server $\mathcal{KS}_j$ with the following key set corresponding to sensor S_i :

$$\mathcal{K}_{i,j}^{\text{server}} = \left\{ (k_i^{\text{sync,old}}, k_i^{\text{sync,new}}), k_{i,j}^{\text{KS-S}}, (k_{i,j}^{\text{emg}}, r_i^{\text{U,emg}}) \right\},$$

the key update is performed as follows.

Server-sensor log. Upon receiving a committed server-sensor log

$$\mathcal{L}_i^{\text{KS-S}} = (ID_{S_i}, \{\Delta_{i,j} \mid j \in [n]\}),$$

key server $\mathcal{KS}_j$ applies the following updates:

$$\begin{aligned} & - k_i^{\text{sync,old}} \leftarrow k_i^{\text{sync,new}} \\ & - k_i^{\text{sync,new}} \leftarrow \text{Upd}(k_i^{\text{sync,new}}) \\ & - k_{i,j}^{\text{KS-S}} \leftarrow (\Delta_{i,j} + \text{Upd}(k_{i,j}^{\text{KS-S}})) \bmod P \end{aligned}$$

Emergency log. Upon receiving a committed emergency log

$$\mathcal{L}_i^{\text{emg}} = (ID_{S_i}, r_i^{\text{U,emg,new}}, \{\text{Enc}_{\text{PK}_j^{\text{KS}}}(k_{i,j}^{\text{emg,new}}) \mid j \in [n]\}) .$$

key server $\mathcal{KS}_j$ performs:

$$\begin{aligned} & - k_{i,j}^{\text{emg}} \leftarrow k_{i,j}^{\text{emg,new}} \\ & - r_i^{\text{U,emg}} \leftarrow r_i^{\text{U,emg,new}} \end{aligned}$$

4.2 Entity Registration

Key Servers, Doctor's Device, and User's Mobile Device Registration

This subsection describes the registration procedures for key servers, doctors' devices, and users' mobile devices. In all cases, the trusted authority $\mathcal{TA}$ is responsible for identity verification and certificate issuance.

Key Server. Upon registration, each key server $\mathcal{KS}_j$ for $j \in [n]$ generates a permanent public-private key pair $(\text{PK}_j^{\text{KS}}, \text{SK}_j^{\text{KS}})$. The trusted authority $\mathcal{TA}$ verifies the identity of each key server and issues a digital certificate binding the server identity to its public key PK_j^{KS} . All key-server certificates are stored in a publicly accessible registry that can be queried by registered users' mobile devices and doctors' devices for certificate verification.

Doctor's Device. Upon registration of a doctor's device D^{Dr} , the device generates a permanent public-private key pair $(\text{PK}^{\text{Dr}}, \text{SK}^{\text{Dr}})$. The trusted authority $\mathcal{TA}$ verifies the doctor's identity and issues a digital certificate binding the doctor's device identity to its public key PK^{Dr} . The resulting certificate is recorded on the replicated ledger accessible to the key servers.

User's Mobile-Device. Similarly, upon registration of a user's mobile device M_i^{user} , the device generates a permanent public-private key pair $(\text{PK}_i^{\text{user}}, \text{SK}_i^{\text{user}})$. The trusted authority $\mathcal{TA}$ verifies the user's identity and issues a digital certificate binding the user identifier to the public key $\text{PK}_i^{\text{user}}$. This certificate is also recorded on the replicated ledger accessible to the key servers.

Implantable Medical Device Registration When a new implantable medical device (sensor) S_i with identifier ID_{S_i} is registered, the trusted authority $\mathcal{TA}$ generates and distributes the initial pre-shared keys to the sensor S_i , the user's mobile device M_i^{user} , and the key servers $\mathcal{KS}$. This is the only phase in which $\mathcal{TA}$ participates; after deployment, no further involvement from $\mathcal{TA}$ is required.

This initialisation step, therefore, consists of generating and storing the appropriate keys on the sensor, the mobile device, and the key servers, as detailed below.

Sensor Key Material. For sensor S_i , $\mathcal{TA}$ generates the following keys: 1) the synchronization key k_i^{sync} , 2) the user-sensor key $k_i^{\text{U-S}}$, and the server-sensor key $k_i^{\text{KS-S}}$. Each key is sampled uniformly at random from a secure key space of size 2^λ , where λ is the system's security parameter. $\mathcal{TA}$ also samples a random value $r_i^{\text{S,emg}}$ to represent the sensor-side randomness contributing to session-key generation in the next emergency event.

$\mathcal{TA}$ then stores in sensor S_i the following key set:

$$\mathcal{K}_i^{\text{sensor}} = \{k_i^{\text{sync}}, k_i^{\text{U-S}}, k_i^{\text{KS-S}}, r_i^{\text{S,emg}}\}.$$

Mobile-Device Key Material. For sensor S_i , $\mathcal{TA}$ sends the user-sensor key $k_i^{\text{U-S}}$ to the corresponding user's mobile device M_i^{user} . This key enables the user's contribution to the session-key computation and is identical to the corresponding key stored in the sensor S_i .

Key-Server Material. For sensor S_i , $\mathcal{TA}$ sends the following keys to every key server $\mathcal{KS}_j$:

1. Synchronization keys ($k_i^{\text{sync,old}}$, $k_i^{\text{sync,new}}$): Key servers maintain the synchronization key of both the previous and current states. This is to ensure maintaining consistent update states with the sensor across all future sessions. For the initialization, the synchronization keys are set as follows: $k_i^{\text{sync,old}} = \emptyset$ and $k_i^{\text{sync,new}} = k_i^{\text{sync}}$.
2. Server-sensor key share $k_{i,j}^{\text{KS-S}}$: The server-sensor key $k_i^{\text{KS-S}}$ is secret-shared across the key servers as

$$k_{i,j}^{\text{KS-S}} = \mathcal{SC}(k_i^{\text{KS-S}}, j),$$

where $\mathcal{SC}$ is the secret-sharing scheme described in Section 4.1.

3. Emergency-access key share $k_{i,j}^{\text{emg}}$ and randomness values ($r_i^{\text{U,emg}}$, $r_i^{\text{S,emg}}$): Used exclusively when the user's mobile device is unavailable and clinical emergency access is required. The emergency-access key is derived from the user-sensor key via a key-derivation function, and each server's share is computed as

$$k_{i,j}^{\text{emg}} = \mathcal{SC}(\mathcal{KDF}(k_i^{\text{U-S}} \parallel r_i^{\text{U,emg}} \parallel r_i^{\text{S,emg}}), j),$$

where $r_i^{\text{U,emg}}$ is a random value sampled by $\mathcal{TA}$ to represent the user-side randomness contributing to the session key generation in the next emergency

event, and $r_i^{S,\text{emg}}$ is identical to the emergency randomness stored in sensor S_i .

Each key server $\mathcal{KS}_j$ stores the the following key set for sensor S_i :

$$\mathcal{K}_{i,j}^{\text{server}} = \left\{ (k_i^{\text{sync,old}}, k_i^{\text{sync,new}}), k_{i,j}^{\text{KS-S}}, (k_{i,j}^{\text{emg}}, r_i^{\text{U,emg}}) \right\}.$$

5 Session-Key Establishment Protocol

This section describes the protocol for establishing a session key between a doctor's device D^{Dr} and sensor S_i , with the assistance of the key servers and the user's mobile device.

- **Protocol initiation.** The doctor's device D^{Dr} samples two nonces $R^{\text{Dr}} = (r_1^{\text{Dr}}, r_2^{\text{Dr}})$, where $r_1^{\text{Dr}}, r_2^{\text{Dr}} \in \{0, 1\}^\lambda$, and sends

$$m_1 = ID_{\text{Dr}} \parallel U_{S_i} \parallel R^{\text{Dr}}$$

to sensor S_i , where U_{S_i} denotes the latest update information and is set to $\emptyset$ if no update is required.

- **Sensor-side update check.** Upon receiving m_1 , sensor S_i processes the update information U_{S_i} . If $U_{S_i} = \emptyset$, no update is performed and the protocol proceeds to the next step. Otherwise, if

$$U_{S_i} = (\text{MAC}_{\text{upd}}, r^{\text{KS,upd}}),$$

the sensor verifies

$$\text{MAC}_{\text{Upd}(k_i^{\text{sync}})}(r^{\text{KS,upd}}) = \text{MAC}_{\text{upd}}.$$

If the verification succeeds, the sensor applies the key-update procedure once; otherwise, it aborts the session.

- **Sensor-side authentication tag generation.** The sensor samples four nonces $R^S = (r_1^S, r_2^S, r_3^S, r_4^S)$, where $r_1^S, \dots, r_4^S \in \{0, 1\}^\lambda$, and computes

$$\text{MAC}_1^S = \text{MAC}_{k_i^{\text{sync}}}(ID_{S_i} \parallel r_1^S \parallel ID_{\text{Dr}} \parallel r_1^{\text{Dr}}),$$

and

$$\text{MAC}_2^S = \text{MAC}_{k_i^{\text{U-S}}}(ID_{S_i} \parallel r_2^S \parallel ID_{\text{Dr}} \parallel r_2^{\text{Dr}}).$$

The sensor sends

$$m_2 = \text{MAC}_1^S \parallel \text{MAC}_2^S \parallel ID_{S_i} \parallel ID_{\text{Dr}} \parallel R^S$$

to the doctor's device and temporarily stores ID_{Dr} , r_1^S , r_3^S , and r_4^S for the current key establishment session

- **Doctor's device request to key servers.** The doctor's device generates an ephemeral key pair $(\text{ePK}^{\text{Dr}}, \text{eSK}^{\text{Dr}})$ and constructs

$$m'_3 = \text{MAC}_1^S \parallel \text{MAC}_2^S \parallel ID_{S_i} \parallel R^S \parallel ID_{\text{Dr}} \parallel R^{\text{Dr}} \\ \parallel \text{ePK}^{\text{Dr}} \parallel \text{Emg},$$

where $\text{Emg} \in \{0, 1\}$ indicates whether emergency access is requested. The doctor signs m'_3 using the long-term private key SK^{Dr} and sends

$$m_3 = m'_3 \parallel \sigma_{\text{SK}^{\text{Dr}}}(m'_3)$$

to the key servers.

- **Leader key-server selection and synchronization.** One of the online key servers is selected as the leader, denoted by $\mathcal{KS}^{\text{leader}}$, to coordinate the current session-key establishment. The leader first verifies the doctor's signature and the corresponding certificate to authenticate the request. It is assumed that the local storage of $\mathcal{KS}^{\text{leader}}$ is synchronized with the latest committed ledger state, and that the following key set associated with sensor S_i is stored locally:

$$\mathcal{K}_{i,\text{leader}}^{\text{server}} = \left\{ (k_i^{\text{sync,old}}, k_i^{\text{sync,new}}), k_{i,\text{leader}}^{\text{KS-S}}, \right. \\ \left. (k_{i,\text{leader}}^{\text{emg}}, r_i^{\text{U,emg}}) \right\}.$$

If the following equality holds,

$$\text{MAC}_1^S = \text{MAC}_{k_i^{\text{sync,old}}} (ID_{S_i} \parallel r_1^S \parallel ID_{\text{Dr}} \parallel r_1^{\text{Dr}}),$$

then the sensor is one update state behind. In this case, the leader key server generates update information for sensor S_i as

$$\text{U}_{S_i} = (\text{MAC}_{\text{upd}}, r^{\text{KS,upd}}),$$

where $r^{\text{KS,upd}} \xleftarrow{\$} \{0, 1\}^\lambda$ is a fresh nonce and

$$\text{MAC}_{\text{upd}} = \text{MAC}_{k_i^{\text{sync,new}}} (r^{\text{KS,upd}}).$$

The leader key server then responds to the doctor's device with **Aborted** $\parallel \text{U}_{S_i}$, indicating that the sensor is out of sync. To proceed, the doctor's device must restart the protocol and include the updated U_{S_i} in the initial message to sensor S_i .

If the following equality holds,

$$\text{MAC}_1^S = \text{MAC}_{k_i^{\text{sync,new}}} (ID_{S_i} \parallel r_1^S \parallel ID_{\text{Dr}} \parallel r_1^{\text{Dr}}),$$

then the sensor and the key servers are synchronized, and the leader key server proceeds to the next step of the protocol.

- **Session key request broadcast.** After successful synchronization, the leader key server $\mathcal{KS}^{\text{leader}}$ generates an ephemeral public-private key pair $(\text{ePK}^{\text{leader}}, \text{eSK}^{\text{leader}})$ for the current session. It then constructs and broadcasts the following message to all key servers over authenticated and encrypted channels:

$$m_4 = ID_{S_i} \parallel \text{Emg} \parallel k_i^{\text{sync,new}} \parallel \text{ePK}^{\text{leader}}.$$

Non-emergency case ($\text{Emg} = 0$). In parallel with broadcasting m_4 , the leader key server constructs

$$m'_5 = \text{MAC}_2^S \parallel r_2^S \parallel ID_{S_i} \parallel r_2^{\text{Dr}} \parallel ID_{\text{Dr}} \parallel r_3^S \parallel \text{ePK}^{\text{Dr}},$$

signs it using its long-term private key, and sends

$$m_5 = m'_5 \parallel \sigma_{\text{SK}^{\text{leader}}}(m'_5)$$

to the user's mobile device M_i^{user} .

Upon receiving m_5 , the user's mobile device verifies

$$\text{MAC}_2^S = \text{MAC}_{k_i^{\text{U-S}}}(ID_{S_i} \parallel r_2^S \parallel ID_{\text{Dr}} \parallel r_2^{\text{Dr}}).$$

If the verification succeeds, the mobile device samples $r_1^{\text{U}} \xleftarrow{\$} \{0, 1\}^\lambda$, derives its session-key contribution

$$k_{\text{session}}^{\text{U-S}} = \mathcal{KDF}(k_i^{\text{U-S}} \parallel r_1^{\text{U}} \parallel r_3^S),$$

and sends

$$m'_6 = \text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{\text{session}}^{\text{U-S}}) \parallel r_1^{\text{U}}$$

to the leader key server, together with a signature, forming

$$m_6 = m'_6 \parallel \sigma_{\text{SK}^{\text{user}}}(m'_6).$$

Meanwhile, each online key server $\mathcal{KS}_j$ responds to m_4 by sending

$$m_{7,j} = ID_{S_i} \parallel \text{Enc}_{\text{ePK}^{\text{leader}}}(k_{i,j}^{\text{KS-S}}) \parallel \emptyset$$

to the leader over an authenticated and encrypted channel.

Emergency case ($\text{Emg} = 1$). If emergency access is requested, the leader key server does not contact the user's mobile device. Instead, each key server $\mathcal{KS}_j$ responds to m_4 by sending

$$m_{7,j} = ID_{S_i} \parallel \text{Enc}_{\text{ePK}^{\text{leader}}}(k_{i,j}^{\text{KS-S}}) \parallel \text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{i,j}^{\text{emg}})$$

to the leader key server over an authenticated and encrypted channel.

- **Distributed encrypted key derivation.** Once the leader key server $\mathcal{KS}^{\text{leader}}$ collects valid responses from at least $t - 1$ other key servers, it combines them with its own stored data to construct the required keys. First, the leader decrypts the received ciphertexts $\text{Enc}_{\text{ePK}^{\text{leader}}}(k_{i,j}^{\text{KS-S}})$ to reconstruct the server-sensor key $k_i^{\text{KS-S}}$. After reconstructing $k_i^{\text{KS-S}}$, the leader computes its updated value $\text{Upd}(k_i^{\text{KS-S}})$. For each $j \in [n]$, it then derives the update token

$$\Delta_{i,j} = \mathcal{SC}(\text{Upd}(k_i^{\text{KS-S}}), j) - \text{Upd}(k_{i,j}^{\text{KS-S}}) \bmod P.$$

The leader submits the following server-sensor log entry to the replicated ledger:

$$\mathcal{L}_i^{\text{KS-S}} = (ID_{S_i}, \{\Delta_{i,j} \mid j \in [n]\}).$$

While the submitted log entry is being committed by the key servers, the leader computes the server-side contribution to the session key.

The leader key server samples fresh nonces $r_1^{\text{KS}}, r_2^{\text{KS}} \xleftarrow{\$} \{0, 1\}^\lambda$ and computes

$$\text{MAC}_1^{\text{KS}} = \text{MAC}_{k_i^{\text{sync}}}(ID_{S_i} \parallel r_1^S \parallel ID_{\text{Dr}} \parallel r_1^{\text{KS}}).$$

The server-side contribution to the session key is then derived as

$$k_{\text{session}}^{\text{KS-S}} = \mathcal{KDF}(k_i^{\text{KS-S}} \parallel r_2^{\text{KS}} \parallel r_4^S).$$

Non-emergency case ($\text{Emg} = 0$). The leader uses the encrypted user contribution $\text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{\text{session}}^{\text{U-S}})$ received in message m_6 from the user's mobile device to homomorphically compute

$$k_{\text{session}}^{\text{KS-U-S,enc}} = \text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{\text{session}}^{\text{U-S}} + k_{\text{session}}^{\text{KS-S}}).$$

The leader sets $r^{\text{U}} = r_1^{\text{U}}$ as included in message m_6 .

Emergency case ($\text{Emg} = 1$). The leader uses the encrypted emergency key shares $\text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{i,j}^{\text{emg}})$ received in messages $m_{7,j}$ from at least $t - 1$ key servers, together with its own emergency share, to homomorphically reconstruct $\text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{\text{session}}^{\text{U-S}})$. It then computes

$$k_{\text{session}}^{\text{KS-U-S,enc}} = \text{Enc}_{\text{ePK}^{\text{Dr}}}(k_{\text{session}}^{\text{U-S}} + k_{\text{session}}^{\text{KS-S}}).$$

In this case, the leader sets $r^{\text{U}} = r_i^{\text{U,emg}}$ from its local storage for sensor S_i .

Once the log entry is committed, all key servers, including the leader, immediately update the key set associated with sensor S_i . Upon log commitment, the leader constructs

$$m'_8 = \text{MAC}_1^{\text{KS}} \parallel k_{\text{session}}^{\text{KS-U-S,enc}} \parallel r_1^{\text{KS}} \parallel r_2^{\text{KS}} \parallel r^{\text{U}},$$

signs it, and sends

$$m_8 = m'_8 \parallel \sigma_{\text{SK}^{\text{leader}}}(m'_8)$$

to the doctor's device.

- **Session-key derivation at the doctor’s device.** Upon receiving message m_8 , the doctor’s device verifies the leader’s signature and decrypts $k_{\text{session}}^{\text{KS-U-S,enc}}$ to recover

$$k_{\text{session}}^{\text{U-S}} + k_{\text{session}}^{\text{KS-S}} \bmod P.$$

It then samples $r_3^{\text{Dr}} \xleftarrow{\$} \{0, 1\}^\lambda$ and derives the final session key as

$$k_{\text{session}} = \mathcal{KDF}(k_{\text{session}}^{\text{U-S}} + k_{\text{session}}^{\text{KS-S}} \parallel r_3^{\text{Dr}}).$$

The doctor’s device subsequently sends the following message to sensor S_i :

$$m_9 = \text{MAC}_1^{\text{KS}} \parallel \text{Emg} \parallel r_1^{\text{KS}} \parallel r_2^{\text{KS}} \parallel r^{\text{U}} \parallel r_3^{\text{Dr}}.$$

- **Session-key derivation at the sensor.** Upon receiving m_9 , sensor S_i verifies MAC_1^{KS} using its stored synchronization key. If verification fails, the session is aborted. Otherwise, the sensor proceeds with session-key derivation by setting

$$r^{\text{S}} = \begin{cases} r_3^{\text{S}}, & \text{if Emg} = 0, \\ r_i^{\text{S,emg}}, & \text{if Emg} = 1. \end{cases}$$

The sensor derives the session key as

$$k_{\text{session}} = \mathcal{KDF}\left(\left(\mathcal{KDF}(k_i^{\text{U-S}} \parallel r^{\text{U}} \parallel r^{\text{S}}) + \mathcal{KDF}(k_i^{\text{KS-S}} \parallel r_2^{\text{KS}} \parallel r_4^{\text{S}}) \bmod P\right) \parallel r_3^{\text{Dr}}\right).$$

The sensor then applies the key-update procedure once.

6 Related Work

Research on the security of implantable medical devices spans multiple directions, driven by the tension between safety, usability, energy constraints, and confidentiality. We briefly review the most relevant lines of work and clarify how our approach differs in scope and goals.

Wireless Attacks and System Analyses. Early work demonstrated that IMDs are vulnerable to practical wireless attacks, including eavesdropping and command injection using software radio techniques [6]. Subsequent analyses confirmed that security weaknesses persist in newer generations of cardiac and neurostimulation devices, often due to proprietary protocols that prioritise power efficiency over cryptographic protection [10, 12]. Attacks on insulin delivery systems further illustrate the risks posed by unauthenticated or weakly protected wireless control channels [8]. A comprehensive systematisation of knowledge shows that many

proposed defences optimise individual aspects such as access control or availability, while leaving long term confidentiality insufficiently addressed [16]. These works motivate the need for stronger cryptographic guarantees under realistic deployment assumptions.

Physiological and Proximity-Based Pairing. A significant body of work explores using the patient’s body or physical proximity as a basis for authentication or key establishment. Heart-to-Heart and related approaches derive shared secrets from physiological signals such as heartbeats [7, 15], while proximity-based schemes use distance bounding or ultrasonic channels to restrict access to nearby devices [13]. Although attractive from a usability perspective, later work shows that physiological signals often lack sufficient entropy and stability for reliable cryptographic use, leading to high false rejection rates in practice [9]. Moreover, remote sensing attacks demonstrate that such signals can be recovered at a distance using commodity cameras, undermining their secrecy assumptions [17]. Our work does not rely on biological or proximity-derived entropy and instead focuses on cryptographic state evolution.

Physical-Layer Defences and External Proxies. Another line of research seeks to secure IMDs by controlling the wireless channel itself, for example through friendly jamming or external wearable guardians [5, 19]. While these mechanisms can raise the bar for opportunistic attackers, they rely on assumptions about adversary capabilities and environmental conditions. In particular, friendly jamming does not provide cryptographic confidentiality guarantees against stronger adversaries equipped with multiple antennas and signal processing capabilities [18]. In contrast, our approach does not depend on physical-layer assumptions or additional wearable hardware, and instead provides cryptographic forward secrecy independent of the attacker’s radio capabilities.

Delegation and Smartphone-Centric Architectures. More recent work recognises the role of patient-held devices and proposes delegating key management and access control to a smartphone acting as a security manager [4]. Such designs enable flexible authorisation and revocation and align well with patient-centric workflows. However, they primarily address the authorisation problem, namely who is allowed to access the IMD, rather than the temporal confidentiality problem. If long-term credentials stored on the phone are compromised, historical IMD communications may still be exposed. Our work builds on the insight that smartphones are useful but fallible participants, and distributes trust to avoid making any single client device a permanent point of failure.

Cryptography on Constrained Medical Devices. Several studies have evaluated the feasibility of cryptographic primitives on medical device hardware. Experimental results on insulin pump platforms show that symmetric cryptography is practical on constrained microcontrollers, while frequent public key operations significantly impact energy consumption and device lifetime [11, 14]. These findings motivate designs that minimise public key usage on the implant. Our

protocol follows this direction by using only symmetric primitives on the IMD, while still providing forward secrecy through state evolution and recovery mechanisms.

Human Values, Safety, and Emergency Access. Finally, qualitative studies emphasise that IMD security mechanisms must respect clinical workflows, patient autonomy, and emergency access requirements [2, 3]. Designs that risk lockout during emergencies or impose burdensome procedures are unlikely to be acceptable in practice. Our work explicitly considers these constraints by supporting recovery from desynchronisation and client device loss, and by enabling controlled and auditable emergency access without undermining long-term confidentiality.

In summary, prior work addresses important aspects of IMD security, including pairing, authorisation, and proximity protection. However, none of these approaches directly target forward secrecy and recovery across the full lifecycle of an implant under realistic compromise assumptions. Our work complements existing designs by focusing on cryptographic session lifecycle security for long-lived, safety-critical medical devices.

7 Conclusion

Implantable Medical Devices operate for many years in environments where device loss, backend compromise, and eventual capture are realistic. In such settings, reliance on long lived secrets exposes sensitive medical data once a single credential is compromised, yet conventional approaches for achieving forward secrecy remain difficult to deploy due to strict energy and safety constraints.

In this work, we presented a symmetric only communication protocol that provides forward secrecy for IMDs while remaining compatible with clinical safety requirements. By evolving cryptographic state using lightweight primitives on the implant and introducing a threshold assisted recovery mechanism, the protocol limits retrospective disclosure while avoiding permanent lockout under desynchronisation or loss of external devices. Our formal analysis and implementation on representative 16 bit and 32 bit microcontrollers demonstrate that forward secrecy and recoverability are feasible for long lived implantable devices without relying on frequent public key operations or fragile physical assumptions.

References

1. Avoine, G., Canard, S., Ferreira, L.: Symmetric-Key Authenticated Key Exchange (SAKE) with Perfect Forward Secrecy. In: CT-RSA (2020)
2. Denning, T., Borning, A., Friedman, B., Gill, B.T., Kohno, T., Maisel, W.H.: Patients, Pacemakers, and Implantable Defibrillators: Human Values and Security for Wireless Implantable Medical Devices. In: ACM International Conference on Human Factors in Computing Systems (CHI). pp. 917–926 (2010)

3. Denning, T., Kramer, D.B., Friedman, B., Reynolds, M.R., Gill, B., Kohno, T.: CPS: Beyond Usability: Applying Value Sensitive Design Based Methods to Investigate Domain Characteristics for Security for Implantable Cardiac Devices. In: Annual Computer Security Applications Conference (ACSAC). pp. 426–435 (2014)
4. Duttagupta, S., Marin, E., Singelée, D., Preneel, B.: HAT: Secure and Practical Key Establishment for Implantable Medical Devices. In: Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy. p. 213–224. CODASPY '23 (2023). <https://doi.org/10.1145/3577923.3583646>
5. Gollakota, S., Hassanieh, H., Ransford, B., Katabi, D., Fu, K.: They Can Hear Your Heartbeats: Non-Invasive Security for Implantable Medical Devices. In: ACM SIGCOMM. pp. 2–13 (2011)
6. Halperin, D., Heydt-Benjamin, T.S., Ransford, B., Clark, S.S., Defend, B., Morgan, W., Fu, K., Kohno, T., Maisel, W.H.: Pacemakers and Implantable Cardiac Defibrillators: Software Radio Attacks and Zero-Power Defenses. In: IEEE Symposium on Security and Privacy (S&P). pp. 129–142 (2008)
7. Hu, C., Cheng, X., Zhang, F., Wu, D., Liao, X., Chen, D.: OPFKA: Secure and Efficient Ordered-Physiological-Feature-Based Key Agreement for Wireless Body Area Networks. In: IEEE International Conference on Computer Communications (INFOCOM). pp. 2274–2282 (2013)
8. Li, C., Raghunathan, A., Jha, N.K.: Hijacking an Insulin Pump: Security Attacks and Defenses for a Diabetes Therapy System. In: IEEE International Conference on e-Health Networking, Applications and Services (Healthcom). pp. 150–156 (2011)
9. Marin, E., Argones Rúa, E., Singelée, D., Preneel, B.: On the Difficulty of Using Patient’s Physiological Signals in Cryptographic Protocols. In: ACM Symposium on Access Control Models and Technologies (SACMAT). pp. 113–122 (2019)
10. Marin, E., Singelée, D., Garcia, F.D., Chothia, T., Willems, R., Preneel, B.: On the (in)Security of the Latest Generation Implantable Cardiac Defibrillators and How to Secure Them. In: Proceedings of the 32nd Annual Conference on Computer Security Applications. pp. 226–236. ACSAC '16 (2016)
11. Marin, E., Singelée, D., Yang, B., Verbauwhede, I., Preneel, B.: On the Feasibility of Cryptography for a Wireless Insulin Pump System. In: ACM Conference on Data and Application Security and Privacy (CODASPY). pp. 113–120 (2016)
12. Marin, E., Singelée, D., Yang, B., Volski, V., Vandenbosch, G.A.E., Nuttin, B., Preneel, B.: Securing Wireless Neurostimulators. In: ACM Conference on Data and Application Security and Privacy (CODASPY). pp. 287–298 (2018)
13. Rasmussen, K.B., Castelluccia, C., Heydt-Benjamin, T.S., Capkun, S.: Proximity-Based Access Control for Implantable Medical Devices. In: ACM Conference on Computer and Communications Security (CCS). pp. 410–419 (2009)
14. Rostami, M., Burleson, W., Juels, A., Koushanfar, F.: Balancing Security and Utility in Medical Devices? In: ACM Design Automation Conference (DAC). pp. 1–6 (2013)
15. Rostami, M., Juels, A., Koushanfar, F.: Heart-to-Heart (H2H): Authentication for Implanted Medical Devices. In: ACM Conference on Computer and Communications Security (CCS). pp. 1099–1112 (2013)
16. Rushanan, M., Rubin, A.D., Kune, D.F., Swanson, C.M.: SoK: Security and Privacy in Implantable Medical Devices and Body Area Networks. In: IEEE Symposium on Security and Privacy (S&P). pp. 524–539 (2014)
17. Seepers, R.M., Wang, W., de Haan, G., Sourdis, I., Strydis, C.: Attacks on Heartbeat-Based Security Using Remote Photoplethysmography. *IEEE Journal of Biomedical and Health Informatics* **22**(3), 714–721 (2018)

18. Tippenhauer, N.O., Malisa, L., Ranganathan, A., Capkun, S.: On Limitations of Friendly Jamming for Confidentiality. In: IEEE Symposium on Security and Privacy (S&P). pp. 160–173 (2013)
19. Xu, F., Qin, Z., Tan, C.C., Wang, B., Li, Q.: IMDGuard: Securing Implantable Medical Devices with the External Wearable Guardian. In: IEEE International Conference on Computer Communications (INFOCOM). pp. 1862–1870 (2011)